# Sprint Planning S5

Planeamento da Sprint 5 do Projeto de Laboratório III (Hotel4You)

## Lista de verificação de planeamento do sprint

| Preparação | Reunião | Acompanhamento |
|---|---|---|
| ☐ Retrospetiva da S4 (09/11/2026) revista e ações de melhoria definidas<br>☐ Feedback da Sprint Review da S4 incorporado (menus, importação pelo fornecedor, aprovação de stock, serviços na reserva)<br>☐ Requisitos (RF/RNF) revistos<br>☐ Product Backlog organizado no Trello<br>☐ User stories escritas com critérios de aceitação e codificação | ☐ Definir a meta do sprint<br>☐ Estimar TODOS os itens<br>☐ Escolher os itens do Sprint Backlog<br>☐ Atribuir um responsável a cada story (rodar os papéis: novo Team Leader)<br>☐ Fixar a meta do sprint no Trello | ☐ Daily Scrum no início de cada aula prática e cards do Trello atualizados todos os dias<br>☐ Testes unitários escritos à medida que cada story é desenvolvida<br>☐ Rever pull requests e arrancar a aplicação antes de cada merge<br>☐ Rever as estimativas a meio do sprint<br>☐ Preparar a retrospetiva da S5 (incluindo a organização do grupo) |

## Membros da equipa do sprint

| Nome | Função |
|---|---|
| Simão (1241597) | Developer |
| Guilherme (1251657) | Developer |
| Dinis (1251652) | Developer |
| Leonardo (1251663) | Team Leader |

## Resumo do sprint anterior

| | |
|---|---|
| **Tema do sprint** | Sprint 4 – Check-in e check-out, serviços e extras configuráveis, XML na interface com entrada de stock, e melhorias de interface |
| **Pontos da história** | Estimativas da S4: 32h30m |
| **Resumo** | Retrospetiva de 09/11/2026. A S4 correu bem: tudo foi cumprido a tempo, a organização e a comunicação do grupo voltaram a melhorar e o feedback da S3 foi aplicado. Em demo mostrámos a importação de XML com validação por XSD e entrada em stock, os serviços com preço editável, a paginação, o calendário nas datas, o check-in com atribuição de cartão, o check-out, a regra de só apagar reservas sem check-in e as vistas especiais com valor editável. O docente pediu agora a reorganização dos menus, a importação feita pelo perfil Fornecedor com aprovação pelo gestor ou rececionista antes da entrada em stock, o stock completo, os serviços na reserva e, se houver tempo, a picagem de cartão. Foi também detetado um defeito: alterar o valor de uma vista mudou o preço de reservas já criadas. |

**O que correu bem**

- Tudo concluído a tempo, com sobra para corrigir erros, e boa comunicação dentro da equipa
- Feedback da S3 aplicado: paginação, calendário nas datas, ecrã de reservas mais limpo, serviços e vistas com valor editável
- Check-in com atribuição de cartão, check-out com libertação do quarto e do cartão, estados das reservas e regra de só apagar reservas sem check-in
- Importação do XML com validação por XSD, incluindo rejeição de um ficheiro inválido

**O que não correu bem**

- Alterar o valor de uma vista mudou o preço de reservas já criadas
- Importação feita pelo gestor, com entrada direta em stock e sem aprovação; fornecedor sem dados nem conta própria
- Stock incompleto: sem unidades, preço unitário, taxas e totais; XSD fixo no código
- Serviços ainda não podem ser adicionados à reserva e as reservas não são editáveis
- Menus dispersos e listagem de reservas sem filtro por estado; picagem de cartão por fazer
- Erro de importação dos controllers num commit, resolvido a tempo

## Estado das stories da S4

| ID | Story | Estado no fim da S4 |
|---|---|---|
| S4-01 | Check-in e check-out | Concluída |
| S4-02 | Apagar reservas só sem check-in | Concluída |
| S4-03 | Validação do XML do fornecedor na interface | Concluída; passa a ser feita pelo fornecedor (S5-02) |
| S4-04 | Entrada de stock via XML | Concluída; falta aprovação e stock completo (S5-03 e S5-04) |
| S4-05 | Configuração de serviços e extras | Concluída; falta adicionar à reserva (S5-06) |
| S4-06 | Gestão de vistas especiais | Concluída; defeito do preço nas reservas existentes (S5-09) |
| S4-07 | Melhorias de interface | Concluída; falta reorganizar os menus (S5-01) |

## Agenda

| ID | User story e critérios de aceitação | Estimativa | Prioridade | Responsável |
|---|---|---|---|---|
| S5-01 | Como utilizador da aplicação, quero os menus organizados por tema (Reservas, Fornecedores, Serviços, Stock), para encontrar as funcionalidades sem saltar entre ecrãs.<br>**Critérios de aceitação**<br>☐ Em Reservas estão a criação, edição, cancelamento, apagar, check-in e check-out<br>☐ Existem áreas separadas para Fornecedores, Serviços e Stock<br>☐ O menu fica fixo (lateral ou superior) e os dados abrem no mesmo ecrã<br>☐ Cada perfil só vê as opções a que tem acesso<br>**Codificação**<br>☐ Layout FXML principal com menu fixo e área de conteúdo; controllers sem lógica de negócio<br>☐ Mover as janelas existentes sem alterar o seu comportamento<br>☐ Testar o arranque da aplicação após cada alteração | 10h | Alta | Dinis, Guilherme |
| S5-02 | Como fornecedor, quero ter uma conta própria e importar os meus ficheiros XML, para enviar ao hotel a encomenda a dar entrada.<br>**Critérios de aceitação**<br>☐ O gestor cria o fornecedor com código, nome, morada, código postal e cidade | 5h | Alta | Dinis |

| ID | User story e critérios de aceitação | Estimativa | Prioridade | Responsável |
|---|---|---|---|---|
| | ☐ O fornecedor entra com a sua conta e só importa ficheiros seus<br>☐ A ficha do fornecedor pode ser atualizada a partir dos dados do XML importado<br>☐ Ficheiros inválidos são rejeitados com a linha e o motivo<br>**Codificação**<br>☐ Novo perfil Fornecedor e respetiva janela FXML de importação<br>☐ Campos novos no script SQL e no FornecedorDAO<br>☐ Testes JUnit da importação com ficheiro do próprio e de outro fornecedor | | | |
| S5-03 | Como gestor ou rececionista, quero aprovar ou rejeitar os ficheiros importados pelos fornecedores, para controlar o que dá entrada em stock.<br>**Critérios de aceitação**<br>☐ Os ficheiros importados ficam no estado Pendente<br>☐ O gestor ou rececionista aprova ou rejeita cada ficheiro<br>☐ Só os ficheiros aprovados dão entrada em stock<br>☐ Existe uma listagem de ficheiros pendentes, aprovados e rejeitados<br>**Codificação**<br>☐ Estado do ficheiro e tabela de importações no script SQL; janela FXML de aprovação<br>☐ Regra de aprovação no model/DAO<br>☐ Testes JUnit das transições Pendente, Aprovado e Rejeitado | 2h | Alta | Dinis |
| S5-04 | Como gestor ou funcionário, quero um stock completo com unidades, preços e taxas, para saber o que o hotel tem e o que está disponível para reservas.<br>**Critérios de aceitação**<br>☐ Cada linha guarda produto, quantidade, unidade (caixa e unidades), preço unitário, taxas e total da linha<br>☐ Cada encomenda tem o total das linhas, das taxas e o total com taxas<br>☐ O stock do hotel fica separado do stock disponível (descontando o utilizado nas reservas)<br>☐ A gestão de stock mostra os dois valores<br>**Codificação**<br>☐ Campos novos no XML/XSD e no script SQL; StockDAO atualizado<br>☐ Janela FXML de gestão de stock<br>☐ Testes JUnit dos cálculos de totais e do stock disponível | 4h | Alta | Dinis |
| S5-05 | Como gestor, quero importar ou atualizar o ficheiro XSD pela aplicação, para validar novos formatos sem alterar o código.<br>**Critérios de aceitação**<br>☐ É possível escolher o ficheiro XSD na interface<br>☐ A validação passa a usar o XSD escolhido | 2h30m | Média | Dinis |

| ID | User story e critérios de aceitação | Estimativa | Prioridade | Responsável |
|---|---|---|---|---|
| | ☐ Um XSD inválido é rejeitado com mensagem de erro<br>**Codificação**<br>☐ Janela FXML com FileChooser; a classe de validação recebe o XSD por parâmetro<br>☐ Testes JUnit com um XSD válido e um inválido | | | |
| S5-06 | Como rececionista, quero adicionar serviços e produtos à reserva, indicando a quantidade, para que o custo da estadia fique correto.<br>**Critérios de aceitação**<br>☐ Na criação da reserva é possível escolher serviços e produtos disponíveis<br>☐ É possível indicar a quantidade (por exemplo, duas limpezas)<br>☐ O valor total soma o preço base do quarto, a vista e os extras<br>☐ Os produtos usados reduzem o stock disponível<br>**Codificação**<br>☐ Tabela de ligação entre reserva, serviços e produtos no script SQL<br>☐ Cálculo do valor total no model, não no controller<br>☐ Testes JUnit do total com várias quantidades | 7h | Alta | Leonardo |
| S5-07 | Como rececionista, quero editar uma reserva e ver os seus detalhes, para acrescentar serviços a meio da estadia.<br>**Critérios de aceitação**<br>☐ Os detalhes mostram quarto, hóspede, serviços e extras<br>☐ É possível acrescentar ou remover serviços numa reserva ativa<br>☐ Reservas canceladas ou com check-out feito não podem ser editadas<br>☐ O valor total é recalculado após a edição<br>**Codificação**<br>☐ Janela FXML de edição e detalhes da reserva<br>☐ Regras de edição por estado no model<br>☐ Testes JUnit da edição por estado | 3h | Alta | Simão |
| S5-08 | Como rececionista, quero filtrar as reservas por estado e ver o histórico à parte das ativas, para encontrar uma reserva sem esforço.<br>**Critérios de aceitação**<br>☐ É possível filtrar por estado (reservada, check-in efetuado, check-out efetuado, cancelada)<br>☐ As reservas ativas e o histórico estão em ecrãs separados<br>☐ A listagem continua paginada<br>**Codificação**<br>☐ Consulta filtrada no ReservaDAO com PreparedStatement<br>☐ Janelas FXML separadas para ativas e histórico | 2h | Média | Simão |

| ID | User story e critérios de aceitação | Estimativa | Prioridade | Responsável |
|---|---|---|---|---|
| S5-09 | Como gestor, quero que a alteração do valor das vistas e dos extras só se aplique às reservas seguintes, para não mudar o preço de reservas já feitas.<br>**Critérios de aceitação**<br>☐ O preço da vista e dos extras fica guardado na reserva no momento da criação<br>☐ Alterar o valor de uma vista não altera reservas existentes<br>☐ As reservas novas usam o valor atualizado<br>☐ Os extras permitem valor por noite ou por unidade<br>**Codificação**<br>☐ Campos de preço na tabela Reserva no script SQL<br>☐ Cálculo no model<br>☐ Testes JUnit com uma reserva criada antes e outra depois da alteração | 6h | Alta | Leonardo |
| S5-10 | Como gestor, quero registar a passagem dos cartões nos locais do hotel (quarto, restaurante, piscina), com a hora de entrada, e consultá-la por cartão e por quarto, para controlar os acessos.<br>**Critérios de aceitação**<br>☐ Cada picagem guarda o cartão, o local e a hora<br>☐ O cartão só dá acesso aos locais incluídos na reserva<br>☐ Os funcionários da restauração só registam acesso à cozinha/restaurante<br>☐ O gestor consulta os registos de um cartão e os de um quarto, incluindo o cartão mestre<br>**Codificação**<br>☐ Tabela Passagem no script SQL e PassagemDAO<br>☐ Janela FXML de picagem e de consulta de registos<br>☐ Testes JUnit das regras de acesso por perfil e local | 4h | Média | Simão |

| Responsável | Stories |
|---|---|
| Simão | S5-07, S5-08, S5-10 |
| Guilherme | S5-01 |
| Dinis | S5-01, S5-02, S5-03, S5-04, S5-05 |
| Leonardo | S5-06, S5-09 |

## Detalhes

| | |
|---|---|
| **Data de início** | 10/11/2026 |
| **Data de término** | 16/11/2026 |
| **Tema do sprint** | Reorganização da interface, perfil Fornecedor com aprovação de stock, serviços e produtos nas reservas, e picagem de cartão |
| **Meta do sprint** | No fim da S5, os menus estão organizados por tema, o fornecedor importa os seus ficheiros XML com a sua conta, o gestor ou rececionista aprova ou rejeita antes da entrada em stock, o stock tem unidades, preços e taxas, os serviços e produtos podem ser adicionados e editados nas reservas, o preço das vistas só afeta |

| | |
|---|---|
| | reservas novas, e todas as stories têm Javadoc, testes, estimativas e responsável no Trello. |

### Critérios de codificação

- Estrutura MVC: view → controller → model/DAO; o model não depende de view nem de controller
- GUI em JavaFX com FXML; controllers sem lógica de negócio nem SQL
- Nenhum SQL fora dos DAO; PreparedStatement e ligação numa só classe; sem credenciais no código
- Passwords guardadas encriptadas (hash)
- Nomes: classes em PascalCase, métodos e variáveis em camelCase, constantes em MAIÚSCULAS
- Javadoc em todas as classes e métodos públicos, com @param, @return e @throws quando aplicável
- Entradas lidas e validadas apenas com Utils; exceções tratadas com mensagem clara, sem catch vazio nem System.exit abrupto
- Métodos curtos, uma só responsabilidade, sem código duplicado
- Testes unitários (JUnit) escritos em cada story, na pasta src/test
- Git: branches (feat/fix)-(nº da tarefa)-descrição e commits claros; pull requests revistas e aplicação a arrancar antes do merge
- Atualizar UML (domínio, classes, casos de uso, sequência) e Javadoc em cada story, para a 1.ª entrega

## Possíveis riscos

| Risco | Mitigação |
|---|---|
| O âmbito ser grande para uma sprint e a 1.ª entrega (30/11/2026) estar próxima | Priorizar S5-01 a S5-04, S5-06, S5-07 e S5-09; deixar S5-05, S5-08 e S5-10 para o fim, se sobrar tempo |
| A reorganização dos menus (S5-01) mexer em todas as janelas e partir funcionalidades | Fazer a S5-01 cedo, em pull requests pequenas, testando o arranque a cada alteração |
| O perfil Fornecedor e a aprovação exigirem alterações grandes ao esquema SQL | Script único versionado; todos correm o script do zero após cada alteração |
| Alterar preços voltar a afetar reservas existentes | Guardar o preço na reserva (S5-09) e testar com reservas criadas antes da alteração |
| Erros de importação em commits, como na S4 | Rever pull requests e arrancar a aplicação antes de cada merge |
| Documentação (UML, Javadoc, Scrum) desatualizada na 1.ª entrega | Atualizar em cada story; Team Leader valida antes da Sprint Review |
| Testes unitários continuarem a ser adiados | Cada story só passa a Done com testes; S5-11 em paralelo e revisão entre pares |

# Quadros e retrospetivas de sprint

